

Binary Classification of Edible vs. Poisonous Mushrooms Using Machine Learning (R & Python)

Daniel Camacho Montaña

2025-10-19

Contents

1	Introduction	3
2	Exploración de los datos	3
2.1	Preparación de los datos. Creación de los grupos de entrenamiento y prueba . . .	5
3	Modelos de aprendizaje	6
3.1	k-Nearest Neighbour	6
3.2	Naive Bayes (laplace o no laplace)	9
3.3	Artificial Neural Network	11
3.4	Support Vector Machine	18
3.5	Árboles de decisión: boosting o no boosting	21
3.6	Random forest	23
4	Conclusión y discusión	26
5	Comparación de modelos de clasificación	26

List of Figures

1	Distribución de todas las variables del conjunto de datos <i>mushroom</i>	4
2	PCA: Componentes principales 1 y 2 mostrando la distribución de las clases. . .	5
3	Matriz de Correlación	12
4	Evolución del entrenamiento	15
5	Matriz de confusión de las predicciones	16
6	Evolución del entrenamiento. Modelo 1 vs Modelo 2	17
7	Matriz de confusión de las predicciones. Modelo 2	18

1 Introduction

Los hongos son organismos que, debido a su fisiología, pueden ser comestibles o venenosos, representando un riesgo para la salud si no se identifican correctamente. Para abordar este desafío, se desarrollarán diferentes modelos de aprendizaje que permitirán clasificar hongos según sus características físicas, así como el contexto de su recolección.

El conjunto de datos utilizados es una versión depurada del conjunto de datos *Mushroom* para clasificación binaria disponible en el repositorio de Machine Learning de la UCI [1981]. El conjunto se depuró empleando técnicas como imputación modal, codificación one-hot, normalización Z score y selección de características.

2 Exploración de los datos

El conjunto de datos se encuentra contenido en un documento CSV, el cual se podrá leer mediante la función `read.csv()`. Se debe, para evitar confusión en la lectura de los datos, que los decimales se determinen mediante `;',` y que los datos cuenten con encabezados, que identificarán la variable correspondiente.

Una vez cargados, se podrá ver que tratamos con 9 variables y 10.000 registros, todas detectadas de tipo numérico. Por lo tanto, gracias al contexto de los datos, se observa que incluso las variables categóricas como *stem.color* o *gill.color* se registran como numéricas, representando cada valor numérico una categoría diferente. Esta información es relevante, ya que pese a que algunos algoritmos de clasificación no necesitan de variables categóricas, otros, como las redes neuronales artificiales, necesitan que estén codificadas mediante *one-hot encoding*, ya que en caso contrario no podrán leer bien los datos.

Para profundizar la exploración de los datos, se realizará un análisis descriptivo de todas las variables mediante la función `summary()`, ya que permitirá obtener información como la media de cada variable, así como el rango de valores de cada variable.

Table 1: Resumen estadístico de las variables del conjunto de datos *Mushroom*

Variable	Min.	1er Cuartil	Mediana	Media	3er Cuartil	Max.
cap.diameter	2.0	292.8	526.0	571.0	787.0	1890.0
cap.shape	0.0	2.0	5.0	4.021	6.0	6.0
gill.attachment	0.0	0.0	1.0	2.162	4.0	6.0
gill.color	0.0	5.0	8.0	7.353	10.0	11.0
stem.height	0.00043	0.270	0.597	0.761	1.055	3.835
stem.width	0	417	923	1056	1529	3569
stem.color	0.0	6.0	11.0	8.443	11.0	12.0
season	0.027	0.888	0.943	0.954	0.943	1.804
class	0.0	0.0	1.0	0.549	1.0	1.0

Se puede observar que las variables, debido a que han sido depuradas mediante una normalización Z-score, no todas tienen la misma escala, como podría ser cuando se escalan con otro método (como el escalado min-max). Por ejemplo, la variable *stem.width* tiene una escala de 0-3569, mientras que la variable *cap.diameter* tiene una escala de 2-1890.

Para poder ver cómo se distribuyen los datos en cada variable, se realiza un histograma mediante el paquete `ggplot2`, por lo que primero se modificarán los datos para que tengan un formato "largo".

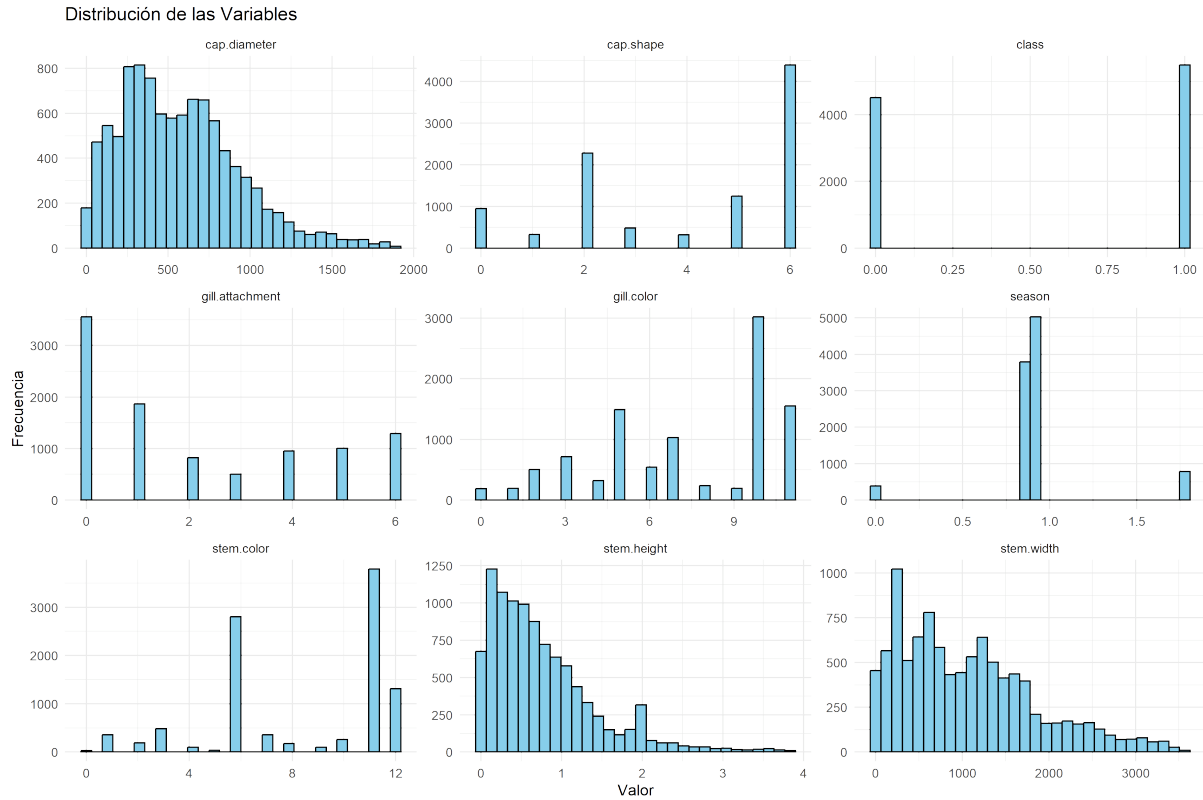


Figure 1: Distribución de todas las variables del conjunto de datos *mushroom*.

Como se ha mencionado anteriormente, las variables consideradas realmente continuas han sido *cap.diameter*, *stem.height* y *stem.width*.

En dichas variables se puede apreciar una distribución en los datos similar a una distribución normal, esperable debido a la transformación de los datos. Aún así, se debe destacar que todas se ven levemente sesgadas negativamente (el centro se ve desplazado hacia la izquierda) como se puede observar en la Figura 1.

Como se ha determinado que los datos se han visto normalizados previamente, se puede realizar un **Análisis de Componentes Principales (PCA)**, de manera que se pueda ver la relación entre dichas variables. No obstante, para el PCA se deben tener en cuenta solo las variables realmente continuas, ya que asume relaciones lineales entre los datos para obtener las componentes principales, pero pese a estar codificadas, las variables categóricas no siguen este principio.

Por este motivo, se creará un subgrupo de datos temporal, *mushroom_num*, que servirá para realizar el PCA.



Figure 2: PCA: Componentes principales 1 y 2 mostrando la distribución de las clases.

Gracias al PCA expresado en la Figura 2, se observa que la CP1 y la CP2 explican el 94.439% de la variabilidad de los datos, pero las clases 0 y 1 se ven mezcladas en el espacio bidimensional creado por ambas CP, por lo que no se puede afirmar que haya una separación lineal entre ellas, al menos no basada en las variables continuas. El hecho de que no se vean separadas claramente no indica que no haya una separación entre las clases, pero requieren de más componentes para poder captar la estructura de los datos, ya que podrían tener patrones no evidentes en este espacio bidimensional.

2.1 Preparación de los datos. Creación de los grupos de entrenamiento y prueba

Un aspecto importante, y común en todos los algoritmos de clasificación, es que necesitan un grupo de datos en el cual basarse para poder realizar las predicciones. Es decir, todos los algoritmos necesitan aprender, de diferentes maneras, cómo se comportan los datos para posteriormente, con datos no vistos, poder determinar a qué clase pertenecen.

Por este motivo, el conjunto de datos debe dividirse, de manera que el algoritmo se entrene mediante unos datos, conociendo el verdadero valor de la clase objetivo, y posteriormente se pueda evaluar su rendimiento aplicándolo a nuevos datos no vistos anteriormente por el algoritmo.

Entonces, se procederá a dividir el dataset original en dos subgrupos: un `train_data` con el 75% de los datos originales para entrenar el algoritmo, y un `test_data` con el 25% restante, para poder evaluar su rendimiento. Como la selección de registros en los grupos se hace de manera aleatoria, se debe establecer una semilla aleatoria para permitir su reproducibilidad.

Después de la partición, se obtendrán el dataset `train_data` con 7500 registros y el dataset `test_data` con 2500.

El algoritmo, una vez entrenado y verificado su rendimiento, tendrá que ser representativo de la población real, no solo de la muestra, ya que en caso contrario se podría padecer de un sobreajuste a los datos vistos. Como la muestra es una representación de dicha población, los subgrupos deben, a su vez, representar la distribución del dataset original. Para evaluar que realmente sean representativos, se estudiarán las proporciones de la variable objetivo en los tres grupos de datos. En la Tabla 2 se muestran las proporciones tanto de la población original como de los subgrupos generados.

Table 2: Proporciones de la variable objetivo en el dataset original y los subgrupos de entrenamiento y prueba

Conjunto	Clase 0	Clase 1
Mushroom	0.4510	0.5490
Train	0.4487	0.5513
Test	0.4580	0.5420

Se puede observar, por lo tanto, que las proporciones en los conjuntos de entrenamiento y prueba son muy similares a las del conjunto original, siendo 45.1% para los comestibles en el dataset original, 44.87% en el grupo de entrenamiento y 45.8% en el grupo de prueba.

Entonces, se puede afirmar que la división de los datos ha mantenido la distribución de las clases objetivo y no resultará en una falta de representación para la creación y entrenamiento del algoritmo.

3 Modelos de aprendizaje

Los algoritmos de aprendizaje son técnicas utilizadas en Machine Learning para permitir que las máquinas aprendan patrones y estructuras a partir de datos sin necesidad de programación explícita. Estos algoritmos procesan grandes volúmenes de datos, ajustando sus parámetros internos con el objetivo de identificar las relaciones entre los datos.

Existen diferentes tipos de algoritmos, como los supervisados y los no supervisados, teniendo cada uno sus propias ventajas y debilidades, y pudiendo realizar gran variedad de tareas.

3.1 k-Nearest Neighbour

El algoritmo k-NN es un método de clasificación supervisada basado en la proximidad entre los datos, ya que funciona identificando los k puntos más cercanos en las características y asignando la clase más frecuente entre esos puntos cercanos. kNN es sencillo y flexible, donde la elección de "k" es un parámetro clave que afectará al rendimiento del modelo.

Una de las necesidades de k-NN radica en que, para que podamos realizar correctamente la predicción (y posteriormente la evaluación del rendimiento), la variable objetivo debe estar en formato *factor*, por lo que tendrá que convertirse.

Una vez todos los datos están en el formato correcto, se procederá a formular el modelo. A diferencia del resto de algoritmos, el modelo kNN no necesita de una etapa de entrenamiento, ya que ejecuta el algoritmo directamente sobre los datos de entrenamiento y obtiene las predicciones proporcionadas por el algoritmo.

Para que el algoritmo de kNN funcione correctamente, se tendrán que codificar las variables categóricas, ya que pese a ser codificadas en una escala, las escalas pueden no ser ordinales, de manera que el algoritmo kNN no podrá calcular bien las distancias entre los datos.

Un método habitual de codificación es el *One-Hot Encoding*, en el que las variables categóricas se dividirán en una matriz de datos. Para poder aplicar dicha codificación, se creará una nueva

función `one_hot_encode` que convierta una variable a factor, obtenga los niveles de dicha variable y cree las columnas necesarias para la codificación. Además, se eliminará la columna original para asegurar la limpieza de los datos.

```
one_hot_encode <- function(data, variable) {
  if (!variable %in% names(data)) {
    stop("La variable especificada no está en el data frame.")
  }
  data[[variable]] <- as.factor(data[[variable]])
  levels_var <- levels(data[[variable]])
  for (level in levels_var) {
    new_col <- paste(variable, level, sep = "_")
    data[[new_col]] <- as.integer(data[[variable]] == level)
  }
  data[[variable]] <- NULL
  return(data)
}
```

Antes de aplicar la función en el conjunto de datos de entrenamiento y prueba, se crearan nuevos `train_data` y `test_data`, de manera que los grupos originales no se vean afectados por la transformación, ya que hay algoritmos que no se verán modificados con esta transformación.

La complejidad del algoritmo kNn reside, entre otros parámetros, en el valor de `k`, ya que determina el número de "vecinos" que considerará antes de atribuir un registro a uno de las clases objetivo. Entonces, el modelo más simple se obtiene con un valor de `k=1`, en el que el algoritmo asociará los datos al siguiente dato que tenga más cerca. Para poder aplicar el algoritmo, solo debemos especificar los datos de entrenamiento, los de prueba, la clase objetivo y el valor de `k`.

```
predictions <- knn(train = train_data_onehot[, -ncol(train_data_onehot)],
  test = test_data_onehot[, -ncol(test_data_onehot)],
  cl = train_data_onehot$class,
  k = 1)
```

Una vez ejecutado, la función `knn()` proporciona las predicciones obtenidas por el algoritmo. Para poder evaluar el rendimiento que ha tenido para clasificar los hongos en comestibles o venenosos, se tiene que generar una matriz de confusión mediante la función `confusionMatrix` del paquete `caret`. Esta matriz contrapondrá los resultados obtenidos por el algoritmo respecto a las clases reales. Los resultados obtenidos fueron:

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	713	440
1	432	915


```

Accuracy : 0.6512
 95% CI : (0.6321, 0.6699)
No Information Rate : 0.542
P-Value [Acc > NIR] : <2e-16

Kappa : 0.2978
```

McNemar's Test P-Value : 0.8126

Sensitivity : 0.6227
Specificity : 0.6753
Pos Pred Value : 0.6184
Neg Pred Value : 0.6793
Prevalence : 0.4580
Detection Rate : 0.2852
Detection Prevalence : 0.4612
Balanced Accuracy : 0.6490

'Positive' Class : 0

Gracias a la matriz de confusión, se obtienen las métricas del rendimiento, de manera que se puede afirmar que, en el caso de $k=1$, se ha obtenido una sensibilidad de 62.2707%, un valor relativamente bajo.

De hecho, el valor de la sensibilidad de este modelo se acerca peligrosamente al azar, ya que al estar cerca del 50%, no habría diferencia entre usar el algoritmo o hacerlo al azar. En cambio, respecto a la especificidad, hemos obtenido un valor de 67.53%. En términos de precisión, hemos obtenido un valor de 65.12%.

En general, el modelo de kNN con $k=1$ no ha resultado muy bueno, pero ha sido un buen punto de inicio en búsqueda de un buen clasificador. Aún así, es posible mejorar el rendimiento de este algoritmo mediante el proceso de reformular el algoritmo con diferentes valores de k .

Aunque a simple vista se pueda pensar que cuanto mayor sea el valor de k , menor será la tendencia a sobreajustarse y mayor el rendimiento, pero este crecimiento no es ilimitado. Si se incrementa de manera desmesurada, sigue existiendo el riesgo de sobreajuste, y además se podría presentar un sesgo que haría menos sensible al modelo a patrones complejos en los datos.

Por lo tanto, se evaluará el modelo kNN con diferentes valores de k , y se contrapondrán evaluando la sensibilidad, especificidad y precisión. Para evitar realizar manualmente el algoritmo para los diferentes valores de k , se creó un bucle que ejecuta el algoritmo y genera las matrices de confusión.

Antes del bucle, primero se debe establecer los diferentes valores de k en `k_values` y un `data.frame results` vacío en el que posteriormente se registrarán los resultados.

En cuanto al bucle, primero se obtendrán las predicciones en base a los diferentes valores de k , para posteriormente realizar la matriz de confusión y, finalmente, se guardarán el rendimiento de los diferentes modelos.

Table 3: Resultados del modelo kNN para distintos valores de k

k	Accuracy	Sensitivity	Specificity
1	0.6512	0.6227	0.6753
3	0.6656	0.6227	0.7018
5	0.6584	0.5965	0.7107
7	0.6636	0.5869	0.7284
11	0.6708	0.5956	0.7343

Se puede observar que, de entre todos los valores de k , el que ha obtenido un valor de sensibilidad máximo ha sido $k=1$, mientras que el que ha obtenido la mejor especificidad ha sido $k=11$. En base a la precisión general del modelo, el que ha tenido un valor máximo ha sido también $k=11$.

Tal y como se comentó anteriormente, el hecho de ampliar al máximo el valor de k no ha resultado en una mejora absoluta del rendimiento del modelo y, de hecho, se ha visto reducida la

sensibilidad respecto a modelos más simples. Este resultado no implica que el algoritmo no sea correcto, pero parece ser que deben tenerse en cuenta otros parámetros para tratar de mejorar el rendimiento.

Por ejemplo, pese a que se ha procedido a optimizar únicamente el parámetro `k`, podrían considerarse otras estrategias para mejorar el rendimiento del modelo, tales como la selección de variables relevantes, el escalado o normalización de los datos, la ponderación de clases desbalanceadas o la combinación con otros algoritmos en un enfoque de ensemble.

3.2 Naive Bayes (laplace o no laplace)

NaiveBayes es un algoritmo de clasificación que se basa en el teorema de Bayes, que utiliza una suposición de independencia condicional entre las características del conjunto de datos. Entonces, este modelo calcula la probabilidad de que una observación pertenezca a cada clase posible basándose en las probabilidades de las características dadas las clases. Este modelo es especialmente útil en la clasificación binaria, como la clasificación comestible-venenoso.

NB puede manejar directamente las variables categóricas, pero estas deben estar en formato de factor para ser leídas correctamente. Anteriormente se pudo ver que, pese a saber que tenemos variables categóricas, estas siguen como formato numérico, por lo que antes de entrenar el modelo se tendrán que modificarlas.

Para poder entrenar el modelo, se requerirá de la función `naiveBayes()` del paquete `e1071`, en la que se debe especificar la variable objetivo (`class`) y el grupo de entrenamiento (`train_data`). A diferencia del modelo anterior, en Naive Bayes sí es necesario entrenar el modelo previamente para que posteriormente pueda realizar las predicciones mediante la función `predict()`.

```
nb_model <- naiveBayes(class ~ ., data = train_data_nb)
predictions <- predict(nb_model, newdata = test_data_nb)
```

Para poder evaluar la capacidad del algoritmo NaiveBayes, se realizará una nueva matriz de confusión, que permitirá ver, de todas las predicciones realizadas por NB, cuáles han resultado correctas.

Confusion Matrix and Statistics

```

      Reference
Prediction  0    1
0  770  419
1  375  936

      Accuracy : 0.6824
      95% CI   : (0.6637, 0.7006)
No Information Rate : 0.542
P-Value [Acc > NIR] : <2e-16

      Kappa   : 0.3622

McNemar's Test P-Value : 0.127

      Sensitivity : 0.6725
      Specificity : 0.6908
Pos Pred Value   : 0.6476
Neg Pred Value   : 0.7140
```

```
Prevalence : 0.4580
Detection Rate : 0.3080
Detection Prevalence : 0.4756
Balanced Accuracy : 0.6816
```

```
'Positive' Class : 0
```

La clasificación con NB ha resultado relativamente buena, con una especificidad de 69.0774908%, pero la sensibilidad no ha sido muy buena (67.25%) y, de hecho, la precisión obtenida del 68.24% es bastante mediocre.

Así como en k-NN se puede mejorar el rendimiento del modelo mediante los valores k , el algoritmo de Naive Bayes también tiene opciones para mejorar su rendimiento.

El **suavizado de Laplace** es una técnica usada en el cálculo de probabilidades cuando las probabilidades tienen una frecuencia de cero. En Naive Bayes es especialmente útil cuando una característica no aparece en los datos de entrenamiento pero sí en los de prueba, haciendo que la probabilidad de esa característica sea cero y guardando ese valor para futuras predicciones.

Para poder aplicar esta técnica, el algoritmo de NB contiene la opción `laplace`, y para activarla solo se debe establecer un valor de 1. Se debe tener en cuenta que, para que el suavizado de Laplace sea efectivo, el algoritmo debe encontrar múltiples datos no vistos anteriormente, ya que en caso contrario, no tiene efecto alguno.

```
nb_model_lp <- naiveBayes(class ~ .,
                           data = train_data_nb,
                           laplace = 1)
```

Para evaluar el rendimiento del algoritmo modificado, recalculemos las predicciones del nuevo modelo y generaremos nuevamente la matriz de confusión utilizando los valores reales. De este modo, podremos analizar su comportamiento y determinar si la aplicación del suavizado de Laplace ha mejorado el desempeño general.

Confusion Matrix and Statistics

```
Reference
Prediction  0   1
0  682 360
1  463 995
```

```
Accuracy : 0.6708
95% CI : (0.652, 0.6892)
No Information Rate : 0.542
P-Value [Acc > NIR] : < 2.2e-16
```

```
Kappa : 0.3323
```

```
McNemar's Test P-Value : 0.0003773
```

```
Sensitivity : 0.5956
Specificity : 0.7343
Pos Pred Value : 0.6545
Neg Pred Value : 0.6824
Prevalence : 0.4580
```

Detection Rate : 0.2728
Detection Prevalence : 0.4168
Balanced Accuracy : 0.6650

'Positive' Class : 0

Posteriormente, construiremos un nuevo *data frame* que recopile las métricas de ambos modelos, lo que permitirá comparar de forma directa sus resultados y cuantificar la mejora obtenida.

Model	Sensitivity	Specificity	Accuracy
<i>Sin Laplace</i>	0.6725	0.6908	0.6824
<i>Con Laplace</i>	0.6716	0.6900	0.6816

Table 4: Comparación del modelo Naive Bayes con y sin suavizado de Laplace.

En el *dataframe* se observa que no se ha modificado el rendimiento notablemente después de aplicar Laplace. De hecho, las métricas se han visto incrementadas muy ligeramente, con la sensibilidad incrementada en un 0.130039%, la especificidad en un 0.106951% y la precisión en un 0.1173709%.

Esto, lejos de ser algo anormal, sucede debido a que en el grupo de prueba no se proporcionan datos no vistos anteriormente en el aprendizaje y, por lo tanto, Laplace no ha tenido efecto alguno en los cálculos de las probabilidades.

Se puede afirmar, entonces, que aplicar Laplace no aporta mejora alguna respecto al modelo sin Laplace, pero no se puede afirmar que no mejoraría con otro grupo de datos no vistos.

3.3 Artificial Neural Network

Las Redes Neuronales Artificiales imitan la estructura del cerebro humano para poder procesar información y resolver todo tipo de problemas complejos.

Como algoritmo de clasificación, destaca su capacidad de manejar problemas multiclase y con variables independientes. Además, entre sus ventajas se incluyen su flexibilidad, la adaptabilidad a los datos y la precisión de sus clasificaciones.

Aún así, una de sus principales desventajas son su alta demanda computacional, su dependencia a la calidad de los datos y la dificultad de interpretar sus resultados. Además de su demanda computacional, la creación de algoritmos de ANN tiene cierta dependencia a Python, ya que los paquetes necesarios están formulados en dicho lenguaje.

Como se dispone de los ficheros, se usará el paquete **pandas**. Para poder cargar el dataset *mushroom*, se establece la ubicación del documento CSV (**file_path**), y se cargará la información con la función **pd.read_csv()**. Del mismo modo que en R, en Python se puede visualizar los primeros registros del dataset con **data.head()**. Los datos cargados contienen tanto datos categóricos como numéricos.

Como se trata con muchos datos y muchas variables, apreciar posibles relaciones (ya sean positivas o negativas) puede resultar complejo. Para poder analizar las relaciones más gráficamente, las columnas numéricas (float y int) se seleccionaran y se calculará la matriz de correlación con **.corr()**, para después crear el gráfico de correlación *heatmap*, y se determinará la paleta de colores como un mapa de color, que indicará si es correlación positiva (rojo) o negativa (azul). Se puede observar la matriz resultante en la Figura siguiente.

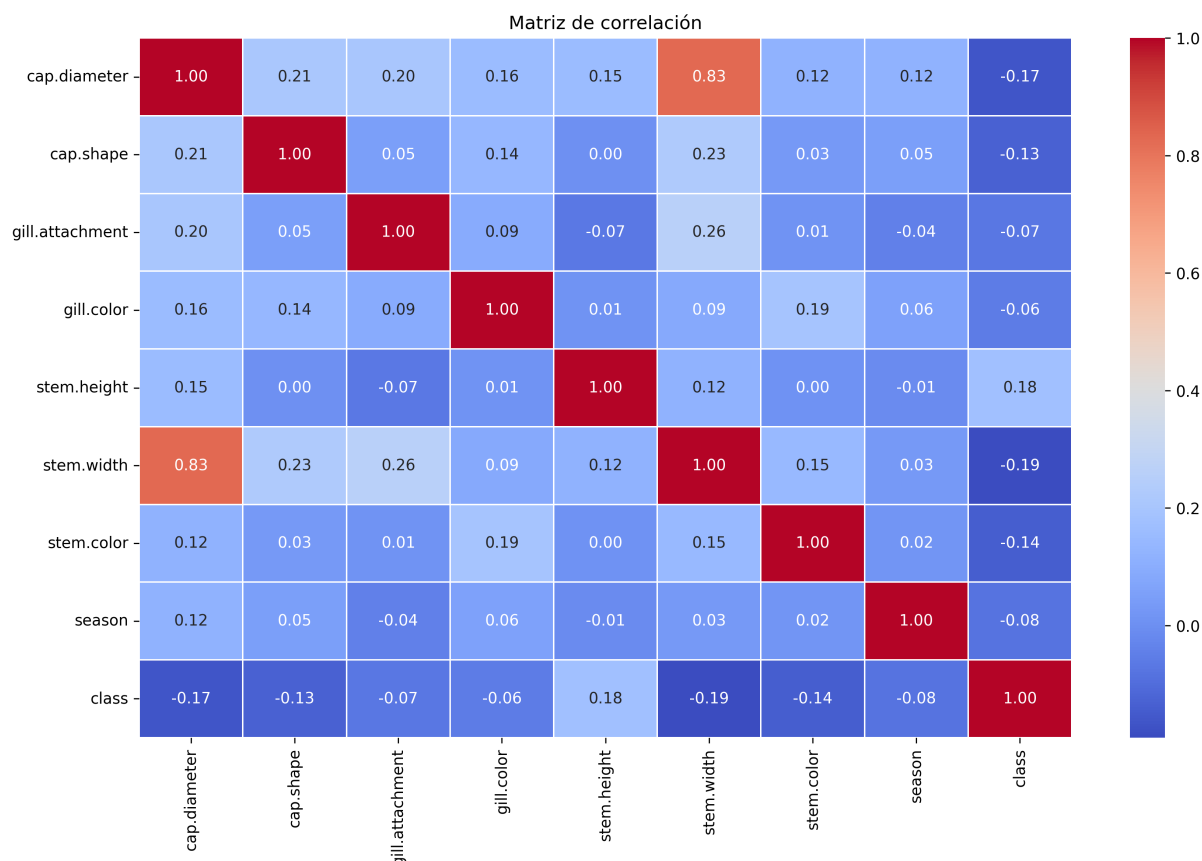


Figure 3: Matriz de Correlación

La matriz de correlación permite ver que, en general, no parece que haya una correlación significativa entre los datos, excepto entre *stem.width* y *cap.diameter*, pero en un sentido biológico, es lógico pensar que el diámetro del sombrero del hongo y el grosor del tallo están positivamente relacionados.

Por lo tanto, en el análisis preeliminar de los datos se puede observar que parecen ser todos de categoría numérica, pero esta información no es del todo correcta. De hecho, gracias al contexto biológico podemos determinar que, pese a que la variable *cap.shape* (por ejemplo) nos indica unos valores numéricos, realmente cada valor nos indica una variable categórica. Del mismo modo, las variables *gill.attachment*, *gill.color* y *stem.color* han resultado categóricas.

Por ende, las redes neuronales artificiales no podrán determinar realmente si, en caso de emplear esta característica para tomar decisiones, serían incorrectas. Un método habitual de codificación es el **One-hot encoding**, de manera que se cree una matriz de datos en que se determine una presencia con "1" y ausencia con "0".

Para poder codificar las variables, se creará la nueva **codification**, que con el input del data set y la variable, cree una matriz de datos con la columna dividida en diferentes columnas codificadas.

```
def codification(data, column_name):
    # Aplicar One-Hot Encoding usando pd.get_dummies
    one_hot = pd.get_dummies(data[column_name], prefix=column_name)

    # Convertir los valores booleanos a enteros (0 y 1)
    one_hot = one_hot.astype(int)

    # Eliminar la columna original
```

```
data = data.drop(column_name, axis=1)

# Añadir las nuevas columnas codificadas al DataFrame
data = pd.concat([data, one_hot], axis=1)

return data
```

Ahora que se ha generado la función que nos permita realizar la codificación, aplicaremos este método a todas las variables que hemos considerado categóricas.

Las variables numéricas, gracias a `describe()`, se ve que tienen un gran rango, y por lo tanto, afectarán al aprendizaje de los datos. Por lo tanto se deben normalizar los datos de alguna manera. Un método habitual de normalización es el **escalado minmax**, en que se establecen los datos en un rango específico, de 0 a 1.

Para ello, primero, se generará la función para el escalado `normalize_minmax()`, para posteriormente aplicarlo a las variables numéricas.

```
def normalize_minmax(data, column):
    data[column] = (data[column] - data[column].min()) / (data[column].max() - data[column].min())
    return data
```

Ahora que se han cargado los datos y se han analizado y transformado, se establecerá una nueva semilla (12345) para poder asegurar la reproducibilidad de los datos.

Para poder crear el algoritmo, entrenarlo y posteriormente evaluarlo, se tiene que dividir el dataset original en dos grupos, un grupo de entrenamiento con el 75% de los datos, y un grupo de prueba con el 25% restante.

El paquete `numpy` contiene una función `.random.seed()` que permite establecer la semilla aleatoria, para posteriormente obtener índices randomizados gracias a la función `np.random.choice()` y, con los índices, generar los grupos de entrenamiento y prueba.

Para verificar que se ha creado correctamente, se imprimirán las dimensiones de los grupos, para confirmar de que no se ha perdido la distribución de las variables o los registros.

Clase	Conjunto completo	Entrenamiento	Prueba
1	0.5490	0.5448	0.5616
0	0.4510	0.4552	0.4384

Table 5: Proporción de clases en el conjunto completo, de entrenamiento y de prueba.

En ambos grupos, las proporciones son relativamente parecidas a los del grupo original, por lo que se verifica que la división ha sido satisfactoria.

De las bibliotecas disponibles capaces de generar redes neuronales en Python, **Tensorflow** es especialmente flexible y capaz de trabajar con datos complejos, incluyendo imágenes o texto. Dentro de `tensorflow`, **keras** permite simplificar la construcción y entrenamiento de los modelos, mientras que **sklearn.metrics** proporciona los resultados de la evaluación mediante una matriz de confusión (como el paquete `caret` en R).

La creación de algoritmos de ANN requiere de la comprensión y establecimiento de ciertos parámetros, los cuales determinarán la capacidad del algoritmo de procesar los datos proporcionados.

Anteriormente se mencionó que las ANN simulan el funcionamiento del cerebro humano. De ese modo, las ANN están configuradas como capas de neuronas conectadas entre ellas, desde la entrada de los datos, hasta los valores de salida. En función de las conexiones, las neuronas se pueden clasificar en tipos como **densas** (conectadas a todas las neuronas de la siguiente capa) o **de convolución** (conectadas a un subconjunto de neuronas de la capa anterior), entre otras.

Además, se debe tener en cuenta que las neuronas pueden ser activadas mediante diferentes funciones de activación, que realizan transformaciones no lineales aplicadas al resultado de la capa antes de pasarlo a la siguiente. Estas funciones permiten, por lo tanto, que la red neuronal aprenda relaciones complejas y aumenta su capacidad de modelado. Existen múltiples tipos de funciones de activación, pero las más comunes son la *Función ReLU* (convierte valores negativos a 0), la **Función Sigmoid**e (convierte los valores de un rango 0-1) y la **Función Softmax** (convierte los valores en probabilidades de total igual a 1).

Finalmente, los algoritmos necesitan de **optimizadores**, que ajustarán los pesos de la red neuronal durante el entrenamiento para que, durante el entrenamiento, minimizar la función de pérdida. De esta manera, la red aprende correctamente ajustando los pesos en cada iteración, mejorando cada vez. Existen tipos diversos de optimizadores, entre los que se encuentran el **Gradiente Descendiente Estocástico** (usa gradientes calculados en pequeñas muestras de datos) y **ADAM** (combina SGD y métodos de gradientes acumulados).

Para poder crear la red neuronal, se usará la función `Sequential()`, seguidas por las diferentes capas de neuronas. Debido a la naturaleza de nuestros datos, se creará un modelo de 2 capas, una capa densa con activación ReLU y una capa densa de salida con activación sigmoide. Esta es una configuración simple para datos binarios (como es el caso de los datos actuales, debido a la clase objetivo). Como mecanismo preventivo ante el sobreajuste, se añadirá una capa de Dropout, que inactivará un porcentaje determinado de neuronas en cada época del aprendizaje.

Una vez diseñada la red, se debe compilar el modelo con el optimizador, y especificar con qué métrica se va a evaluar el rendimiento del modelo (precisión). Para visualizar el resultado del modelo, se empleará la función `summary()` sobre el modelo.

Layer (type)	Output Shape	Param #
Dense	(None, 8)	352
Dropout	(None, 8)	0
Dense_1	(None, 1)	9

Table 6: Resumen de capas y parámetros del modelo.

Total params: 361 (1.41 KB), **Trainable params:** 361 (1.41 KB), **Non-trainable params:** 0 (0.00 B)

Se puede observar que existen 3 capas de neuronas, y la columna Param indica los parámetros que se aprenderán en cada capa de la red.

Hasta el momento, no se ha entrenado el modelo con los datos, solo se ha diseñado para que pueda procesar los datos según su estructura. Para poder entrenarlo, se recurrirá a la función `.drop()`, en que se introduce el conjunto de datos de entrenamiento, la clase objetivo y la cantidad de épocas que ejecutará la red.

Un aspecto no mencionado anteriormente, pero igual de relevante en las redes neuronales artificiales es el efecto del **batch**, que es el tamaño de los lotes en que se dividirá el conjunto de datos de entrenamiento pre-evaluación de los pesos, y el **validation_split**, que especificará el porcentaje del conjunto de datos de entrenamiento que se usará como validación.

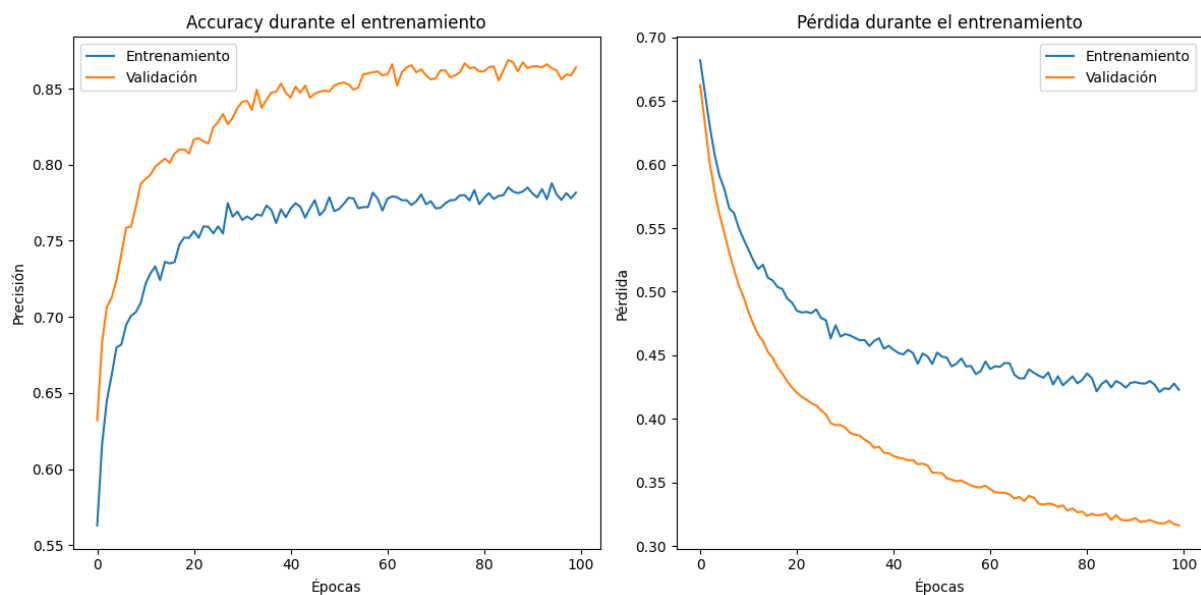


Figure 4: Evolución del entrenamiento

El gráfico de precisión fluctúa un poco (pero podría ser debido al batch), la precisión de validación es bastante más estable, y de hecho se atasca en un valor relativamente alto. Esta estabilidad frente a la variabilidad del entrenamiento podría ser consecuencia del dropout, que ayuda a regularizar el modelo.

Respecto al gráfico de pérdida, tanto el entrenamiento como la validación desciende constantemente, indicando que está aprendiendo y no hay un aparente sobreajuste. La pérdida de validación es más suave y estable que el entrenamiento, lo cual es una buena señal.

Por lo tanto, se puede decir que el dropout funciona como regularización, ya que no hay un sobreajuste a los datos de entrenamiento, y se mantiene un aprendizaje estable.

Hasta el momento, hemos comprobado como funciona el modelo con los datos vistos `train_data`, pero para verificar que el modelo tiene capacidad de generalizar su proceso a datos no vistos, se debe contraponer las predicciones resultantes del modelo con verdaderos datos no vistos anteriormente.

Por lo tanto, se deben usar las características de `test_data` para obtener sus predicciones, y se evaluará el rendimiento del modelo mediante una matriz de confusión. De este modo, se obtendrán los valores de class reales vs los valores predichos.

Para empezar, se visualizan los valores de *loss* y *accuracy* mediante la función `evaluate()`. Entonces, en el modelo de 2 capas (una densa, y una de salida) hemos obtenido una precisión del 89.78% y una pérdida del 30.81%.

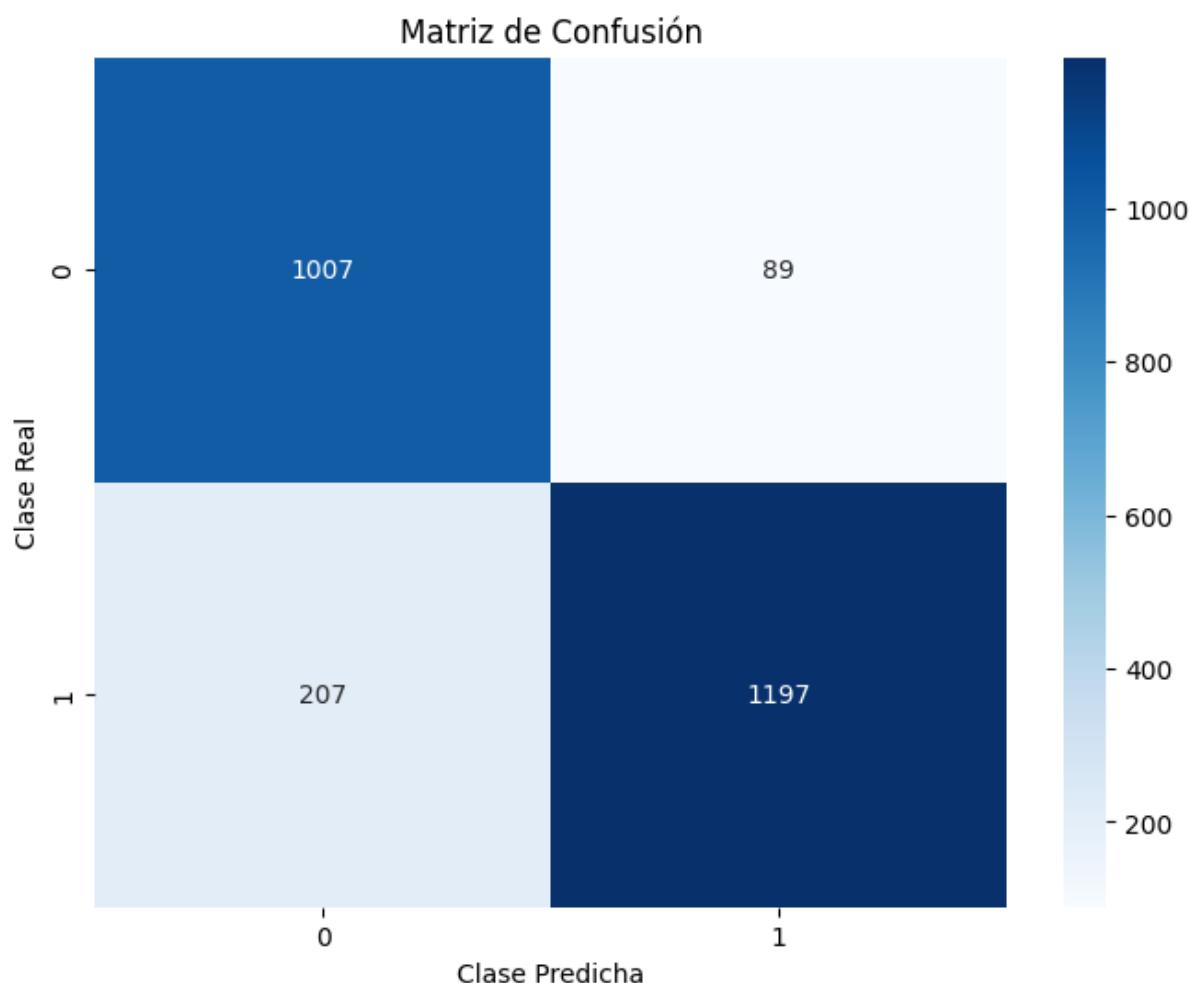


Figure 5: Matriz de confusión de las predicciones

Por lo tanto, se puede observar que en general tienen un rendimiento bastante bueno, con una precisión del 86,89%, una especificidad del 92,61% y una precisión general del 93,77%.

Son muy buenos valores, y se puede decir que el modelo de red neuronal artificial de 2 neuronas es muy eficiente en la clasificación de hongos con las características que se han proporcionado.

Seguidamente, se realizaron las predicciones, especificando `int32` para que las predicciones sigan una clasificación binaria, y se contrapondrán los datos predichos vs reales. Para hacer más visual el resultado de la matriz de confusión, se empleará un heatmap.

Como todos los algoritmos de clasificación, las ANN tienen opciones y mecanismos de mejora del rendimiento. Como se mencionó anteriormente, las redes ANN tienen una configuración bastante compleja, pero la manera más simple de mejora suele ser añadir más capas neuronales en el modelo.

Este mecanismo tiene sus ventajas, ya que permite aprender relaciones no lineales y complejas en los datos, así como construir representaciones jerárquicas de los datos. Aún así, más capas implica también más parámetros, lo que puede llevar a un sobreajuste problemático a los datos de entrenamiento, perdiendo la capacidad de clasificación. Además, también tendrá un mayor coste computacional, y hasta podría afectar a los gradientes hasta volverlos extremadamente pequeños (vanishing).

Para mejorar el modelo anterior, se añadirá una sola capa densa de 10 nodos, con una capa de dropout para tratar de regularizar el sobreajuste.


```

model2 = Sequential()
model2.add(Dense(10, activation='relu', input_shape=(train_data.shape[1] - 1,))) # Capa de
model2.add(Dropout(0.5))
model2.add(Dense(10, activation='relu')) # Segunda capa oculta
model2.add(Dropout(0.5))
model2.add(Dense(1, activation='sigmoid')) # Capa de salida

```

En la Figura siguiente se puede observar el resumen de la arquitectura del modelo.

Layer (type)	Output Shape	Param #
Dense_2	(None, 10)	440
Dropout_1	(None, 10)	0
Dense_3	(None, 10)	110
Dropout_2	(None, 10)	0
Dense_4	(None, 1)	11

Table 7: Resumen de capas y parámetros del modelo.

Total params: 561 (2.19 KB), **Trainable params:** 561 (2.19 KB), **Non-trainable params:** 0 (0.00 B)

Este nuevo modelo tiene ahora 3 capas de aprendizaje y 2 capas de dropout, y los parámetros que se aprenden en cada capa son cada vez menores, lo que puede llevar a un aprendizaje más preciso.

Para evaluar este nuevo modelo, se compilará con un optimizador ADAM y se entrenará con los datos de entrenamiento.

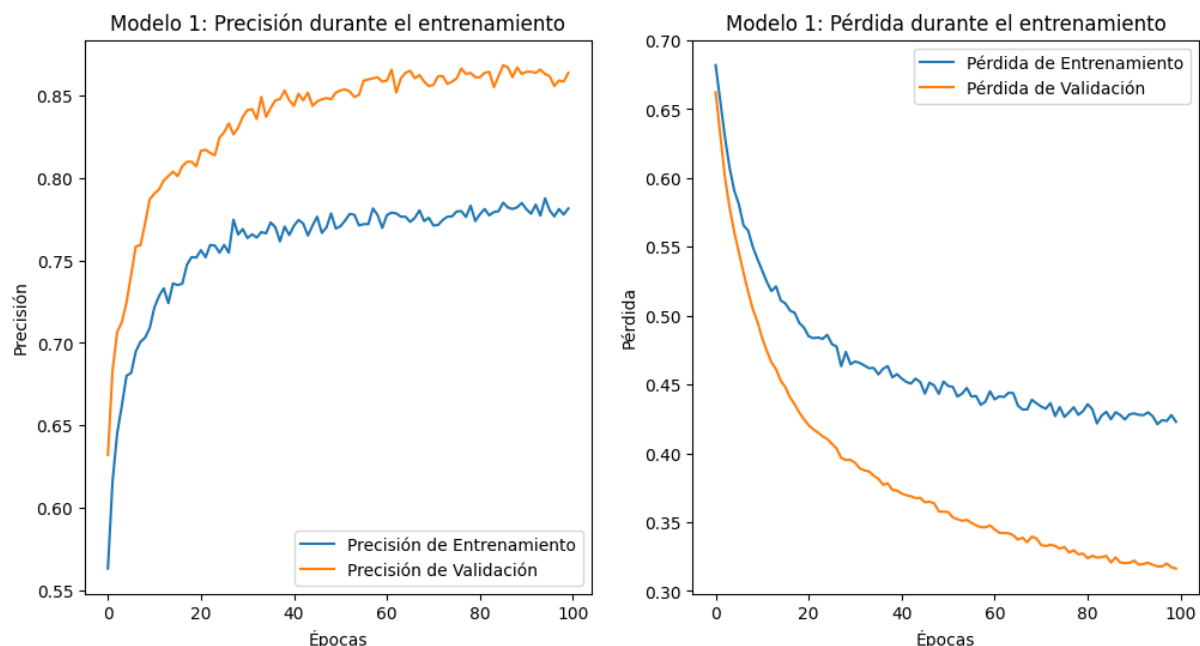


Figure 6: Evolución del entrenamiento. Modelo 1 vs Modelo 2

Gracias a los gráficos, se puede afirmar que la distancia entre la precisión y la validación, así como el entrenamiento y la validación, se han visto reducidas. Esta diferencia indica que el nuevo modelo parece estar aprendiendo mejor como es la relación entre los datos. Además, los valores de precisión y pérdida son ligeramente mejores que en el modelo anterior.

Para poder evaluar realmente el rendimiento del nuevo modelo, se requieren los valores de sensibilidad, especificidad y precisión.

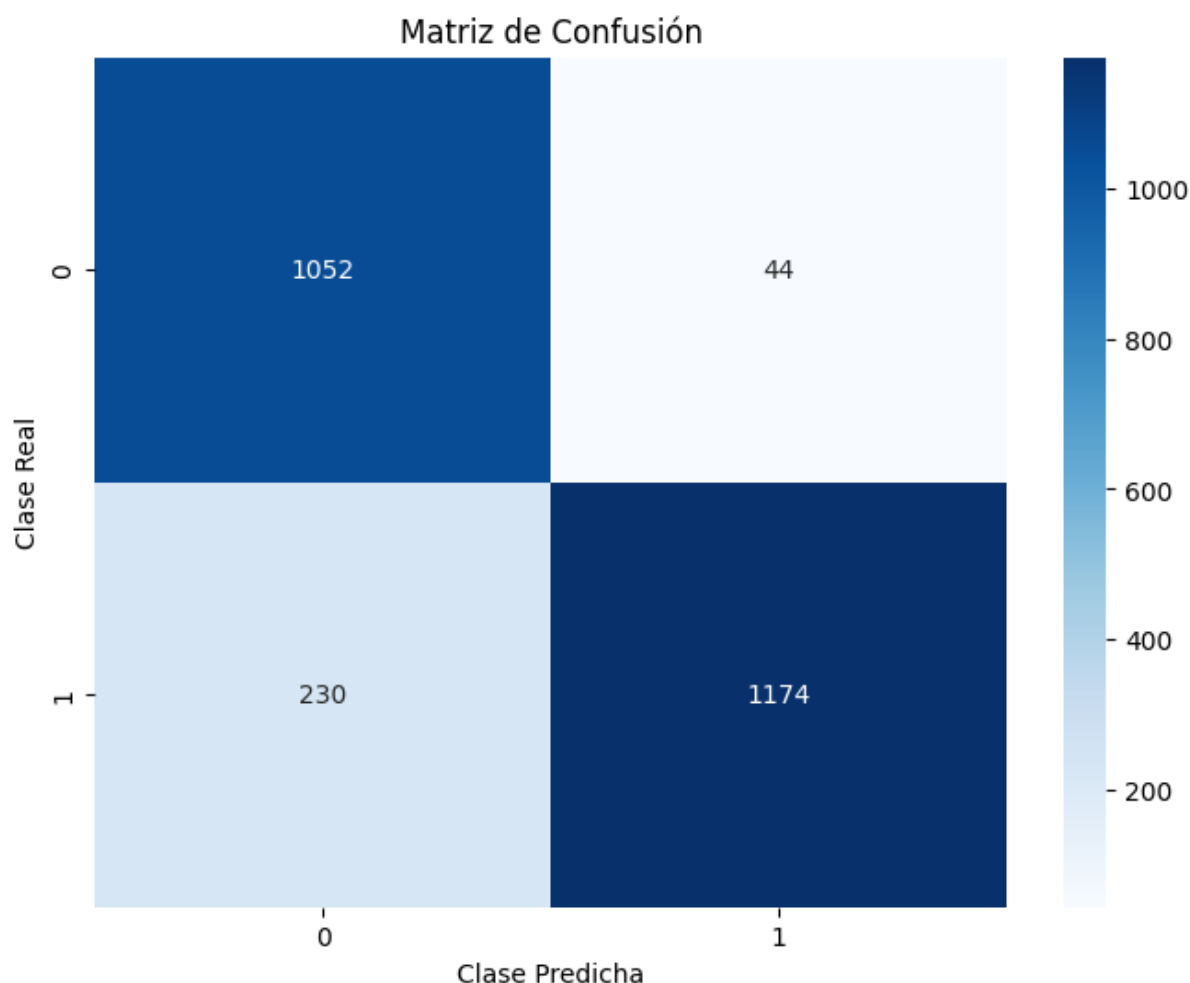


Figure 7: Matriz de confusión de las predicciones. Modelo 2

Pese a ampliar la red neuronal con una capa neuronal densa, se observa que la precisión ha descendido desde un 93,77% a un 85,92%, por lo que el añadir una capa más no solo no ha mejorado su rendimiento, si no que todo lo contrario. Respecto a la sensibilidad, también ha descendido a un 77,99%, pero la especificidad, en cambio, ha incrementado hasta un 96,08%.

Al añadir más capas y nodos puede hacer el modelo más flexible y capaz de captar relaciones complejas, pero en este modelo, el hecho de añadir las parece causar un sobreajuste a los datos de entrenamiento. Esto puede ser porque los modelos más complejos requieren más datos para entrenarse correctamente sin sobreajustarse.

Por lo tanto, debido a nuestros datos, se decantó más por el diseño más simple, el cual ha obtenido un mejor rendimiento en general.

3.4 Support Vector Machine

El **Support Vector Machine (SVM)** es un algoritmo de aprendizaje supervisado usado para la clasificación y regresión. El objetivo principal es encontrar un hiperplano en un espacio de características que separe las diferentes clases de datos con la distancia entre hiperplano y los puntos más amplia posible. Los SVM emplean el **kernel trick** para mapear los datos a un espacio de mayor dimensión y así poder separar linealmente los datos.

Para implementar este modelo, se utilizará la función `svm()` del paquete `e1071`. Para el entrenamiento, el algoritmo necesita la variable objetivo (`class`) y el conjunto de entrenamiento. Como el SVM utiliza cálculos basados en distancias, la opción `scale` permite escalar los datos,

optimizando los cálculos del modelo y mejorando la estabilidad numérica del mismo. También es importante tener en cuenta que el algoritmo SVM no puede gestionar las variables categóricas como *factor*, y deben ser codificadas con técnicas como *One-Hot Encoding*.

El modelo más simple de SVM es el modelo SVM lineal, el cual crea una simple hiperplano lineal que dividirá los datos, pero existen opciones más avanzadas. Además, se activará la función de escalado, ya que de esta manera, se equilibrarán los rangos en los datos.

```
svm_lineal <- svm(class ~ .,
  data = train_data_onehot,
  kernel = "linear",
  scale = TRUE)
```

Gracias a la función `summary()`, se puede ver que el modelo ha generado un total de 4401 vectores de soporte, es decir, son los puntos más cercanos al hiperplano lineal que determinarán la posición de este. Estos puntos son los más importantes para construir el modelo, ya que definen el margen del hiperplano.

Para evaluar el modelo, se realizarán predicciones en base a los datos no observados de `test.data`. De esta manera, y gracias a una matriz de confusión, se puede ver cuán efectivo ha sido este algoritmo.

Confusion Matrix and Statistics

```

      Reference
Prediction  0   1
0  863 377
1  282 978

      Accuracy : 0.7364
      95% CI   : (0.7187, 0.7536)
No Information Rate : 0.542
P-Value [Acc > NIR] : < 2.2e-16

      Kappa : 0.4724

McNemar's Test P-Value : 0.0002505

      Sensitivity : 0.7537
      Specificity : 0.7218
Pos Pred Value : 0.6960
Neg Pred Value : 0.7762
Prevalence : 0.4580
Detection Rate : 0.3452
Detection Prevalence : 0.4960
Balanced Accuracy : 0.7377

      'Positive' Class : 0
```

Por lo tanto, con un kernel lineal, se han obtenido una sensibilidad de 75.371179% y una especificidad de 72.18%, que son relativamente buenos. En general, se ha obtenido una precisión del 73.64%.

Estos no son unos malos resultados, pero tienen un gran margen de mejora. Podría decirse que el hiperplano lineal generado ha podido dividir de manera satisfactoria los datos, pero no ha podido capturar totalmente la relación entre ellos, ya que no parecen tener una partición lineal.

Una manera habitual de mejorar el rendimiento es modificar el tipo de kernel, alterando el cálculo del hiperplano. Además del kernel lineal, existen variantes como el *polinómico*, **sigmoide** o *gaussiano*.

En concreto, el **kernel gaussiano (radial)** realiza una transformación no lineal al proyectar los datos en un espacio de características de mayor dimensión. Esto, dado que se ha verificado que una transformación lineal no ha resultado lo suficientemente buena, permitirá manejar relaciones más complejas entre los datos, ya que el espacio proyectado tiene una curvatura que facilita la separación no lineal.

Para poder aplicar el kernel gaussiano, se debe modificar la opción `kernel` de `lineal` a `radial`.

```
svm_gauss <- svm(class ~ .,
  data = train_data_onehot,
  kernel = "radial",
  scale = TRUE)
```

Con un kernel radial, ahora los vectores de soporte se han visto modificados a 2542, lo cual es habitual, ya que el kernel radial eleva los datos a un hiperplano de mayor dimensión, siendo más complejo, pero capaz de ajustarse más eficazmente a los datos incluso cuando no son linealmente separables.

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	1044	62
1	101	1293

Accuracy : 0.9348
 95% CI : (0.9244, 0.9442)
 No Information Rate : 0.542
 P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.8683

Mcnemar's Test P-Value : 0.002917

Sensitivity : 0.9118
 Specificity : 0.9542
 Pos Pred Value : 0.9439
 Neg Pred Value : 0.9275
 Prevalence : 0.4580
 Detection Rate : 0.4176
 Detection Prevalence : 0.4424
 Balanced Accuracy : 0.9330

'Positive' Class : 0

En el nuevo modelo, se observa que la nueva sensibilidad se ha visto incrementada a un 91.18%, mientras que la especificidad ahora es de 95.42% y la precisión es de 93.48%.

Para poder comparar los resultados de ambos modelos, se visualizarán ambos rendimientos en un **data frame** (**results_svm**) con sus métricas de rendimiento.

Model	Accuracy	Sensitivity	Specificity
Linear	0.7364	0.7537	0.7218
Radial	0.9348	0.9118	0.9542

Table 8: Métricas de rendimiento de los modelos SVM lineal y radial.

Por lo tanto, se puede observar que gracias a la comparación, que los valores máximos de los tres parámetros han sido logrados con el modelo radial, obteniendo una mejora del 26.94188% en la precisión, una mejora del 20.97335% en la sensibilidad y un aumento del 32.20859% en la especificidad.

Se puede afirmar, entonces, que el modelo radial ha sido capaz de capturar patrones más complejos en los datos que el modelo lineal no pudo. Esta mejora no indica que el modelo lineal sea intrínsecamente peor que un modelo radial, sino que el modelo radial funciona mejor con estos datos, debido a que es más flexible y captura relaciones más complejas entre ellos.

3.5 Árboles de decisión: boosting o no boosting

Un método de clasificación ampliamente usado son los árboles de decisión, un método que divide los datos en subconjuntos basados en sus características, utilizando reglas de decisión simples individuales. El árbol resultante será generado en modo jerárquico, donde los nodos representarán una prueba sobre una característica, y las "hojas" serán las clases obtenidas a partir de dichas decisiones.

Este modelo es fácil de interpretar y permite manejar datos no lineales. Para poder crear el algoritmo, emplearemos la función `C5.0()` del paquete `C50`. Esta función creará un árbol (`trials = 1`) a partir de las características (`train_data[-9]`) y la clase objetivo (`train_data$class`).

A diferencia del resto de algoritmos vistos hasta ahora, los árboles de decisión son lo suficientemente flexibles como para no necesitar transformaciones en los datos más allá de la normalización, de manera que pueden captar las variables categóricas sin necesidad de codificación.

```
model_c50<-C5.0(train_data[-9],
               train_data$class,
               trials = 1,
               costs= NULL)
```

El árbol generado tiene un tamaño de 194, lo que significa que tienen un total de 194 nodos, incluyendo tanto los nodos de decisión (internos) como los nodos hoja (clases). Un árbol tan grande podría indicar que es bastante complejo y puede ser un indicio de sobreajuste, por lo que se debe evaluar el rendimiento del modelo.

Es decir, se realizarán predicciones en base a los datos de prueba para observar, en datos no vistos, la capacidad de generalización del modelo. Estas predicciones se evaluarán con una Matriz de confusión, mostrada a continuación.

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	1090	47

1 55 1308

```
Accuracy : 0.9592
95% CI : (0.9507, 0.9666)
No Information Rate : 0.542
P-Value [Acc > NIR] : <2e-16

Kappa : 0.9178

McNemar's Test P-Value : 0.4882

Sensitivity : 0.9520
Specificity : 0.9653
Pos Pred Value : 0.9587
Neg Pred Value : 0.9596
Prevalence : 0.4580
Detection Rate : 0.4360
Detection Prevalence : 0.4548
Balanced Accuracy : 0.9586

'Positive' Class : 0
```

El árbol de decisión ha obtenido valores muy buenos tanto en sensibilidad (95.20%), como en especificidad (96.53%) y hasta en la precisión general (95.92%).

Estos valores han resultado ser extremadamente superiores en comparación con algunos de los modelos planteados anteriormente. Aun así, como el resto de algoritmos, los árboles de decisión tienen sus propias herramientas de mejora. La manera más sencilla de mejorar este tipo de algoritmo es empleando un *boosting*, el cual creará una secuencia de árboles donde cada nuevo modelo corrige los errores cometidos por los árboles anteriores, mejorando progresivamente y, por ende, tomando mejores decisiones.

Para poder aplicar *boosting*, se debe especificar la opción `trials` con el número de árboles que se quiere emplear.

```
model_c50_boost<-C5.0(train_data[-9],
  train_data$class,
  trials = 10,
  costs= NULL)
```

Como ahora no se ha generado un solo árbol de decisión, se debe evaluar el tamaño promedio de los árboles, teniendo en este caso un tamaño medio de 144.1. Se puede observar, entonces, que el tamaño promedio ha resultado menor que el del árbol único de decisión, lo que permite intuir que los árboles han resultado más efectivos en sus decisiones.

Para determinar si, tal y como se intuye por el tamaño de los árboles, el modelo con *boosting* es superior al modelo con un solo árbol, se calcularán las predicciones del nuevo modelo y se visualizarán las métricas de rendimiento.

Confusion Matrix and Statistics

```
Reference
Prediction 0 1
```

```

0 1117 21
1 28 1334

Accuracy : 0.9804
95% CI : (0.9742, 0.9855)
No Information Rate : 0.542
P-Value [Acc > NIR] : <2e-16

Kappa : 0.9605

McNemar's Test P-Value : 0.3914

Sensitivity : 0.9755
Specificity : 0.9845
Pos Pred Value : 0.9815
Neg Pred Value : 0.9794
Prevalence : 0.4580
Detection Rate : 0.4468
Detection Prevalence : 0.4552
Balanced Accuracy : 0.9800

'Positive' Class : 0

```

Por lo tanto, se observa que el nuevo modelo ha obtenido una sensibilidad del 97.55%, una especificidad del 98.45% y una precisión general de 98.04%.

Para poder determinar cuál de los dos modelos es realmente mejor, se creará un **data frame** que permita comparar las métricas de ambos modelos.

Model	Accuracy	Sensitivity	Specificity
Sin boosting	0.9592	0.9520	0.9653
Con boosting	0.9804	0.9755	0.9845

Table 9: Comparación de las métricas de rendimiento entre el modelo C5.0 sin *boosting* y con *boosting*.

Gracias a la comparación, se detecta que los valores máximos de los tres parámetros han sido obtenidos con el modelo con *boosting*, obteniendo una mejora del 2.2101751% en la precisión, una mejora del 2.4770642% en la sensibilidad y un aumento del 1.9877676 en la especificidad.

No parece una mejora sustancial, pero teniendo en cuenta que el rendimiento del modelo de un solo árbol ya era bastante bueno, la mejora hace que la clasificación sea casi perfecta, con una tasa de error mínima de 1.96%.

3.6 Random forest

Como se ha visto anteriormente, aumentar la cantidad de árboles ha resultado en un mejor rendimiento del modelo. Siguiendo este principio, los *Random Forest* son un algoritmo de clasificación que utiliza múltiples árboles de decisión para mejorar la precisión. Cada árbol se entrena con un subconjunto aleatorio de los datos de entrenamiento y de las características, reduciendo así el sobreajuste.

A diferencia de un árbol único, incluso con *boosting*, el modelo obtenido mediante un *Random Forest* genera un modelo más robusto y preciso al combinar las predicciones de varios árboles independientes.

Este modelo se puede implementar mediante la función `randomForest()` del paquete `randomForest`, y requiere de las características de entrenamiento y de la clase objetivo (como los árboles de decisión). Además, la función permite, mediante la opción `ntree`, establecer la cantidad de árboles generados en el bosque.

```
model_forest_50<-randomForest(train_data[-9],
                              train_data$class,
                              ntree=50)
```

Los modelos obtenidos con *Random Forest* nos proporcionan una métrica de evaluación conocida como estimación *OOB* (*Out-Of-Bag*). Debemos tener en cuenta que, en este tipo de algoritmo, no todos los árboles ven los mismos datos, ya que los *Random Forest* aplican un proceso de *bootstrap* en el que cada árbol se entrena con un subgrupo de datos del conjunto de entrenamiento. Este proceso nos proporciona una tasa de error de clasificación en las muestras no utilizadas.

El modelo con $n = 50$ ha permitido obtener un estimador *OOB* del 2.2%. Este valor no es un indicador directo del rendimiento del modelo, ya que para poder evaluar qué tan bien funciona se debe obtener las predicciones del modelo con el conjunto de datos de prueba.

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	1116	22
1	29	1333

Accuracy : 0.9796
95% CI : (0.9733, 0.9848)

No Information Rate : 0.542
P-Value [Acc > NIR] : <2e-16

Kappa : 0.9589

McNemar's Test P-Value : 0.4008

Sensitivity : 0.9747
Specificity : 0.9838
Pos Pred Value : 0.9807
Neg Pred Value : 0.9787
Prevalence : 0.4580
Detection Rate : 0.4464
Detection Prevalence : 0.4552
Balanced Accuracy : 0.9792

'Positive' Class : 0

El bosque ha obtenido valores muy buenos tanto en sensibilidad (97.47%), como en especificidad (98.38%) y hasta en la precisión general (97.96%).

Este rendimiento es bastante bueno, con una precisión cercana a la perfección, pero con cierto margen de mejora. Para ello, se puede ampliar la cantidad de árboles creados en el bosque, de manera que se duplique el número de árboles del bosque original.


```
model_forest_100<-randomForest(train_data[-9],
                                train_data$class,
                                ntree=100)
```

Como era de esperar, el estimador *OOB* se ha visto reducido a un 2.04%. Este descenso es habitual, ya que cuanto mayor es el bosque, menor es el riesgo de sobreajuste y, por lo tanto, las predicciones tienden a ser más confiables, reduciendo el valor del estimador.

Para poder evaluar si el modelo más grande es mejor, se obtendrán las predicciones del nuevo modelo y se evaluará su rendimiento a través de una matriz de confusión.

Confusion Matrix and Statistics

```

      Reference
Prediction  0    1
      0 1117   20
      1   28 1335

      Accuracy : 0.9808
      95% CI   : (0.9746, 0.9858)
No Information Rate : 0.542
P-Value [Acc > NIR] : <2e-16

      Kappa : 0.9613

McNemar's Test P-Value : 0.3123

      Sensitivity : 0.9755
      Specificity : 0.9852
Pos Pred Value : 0.9824
Neg Pred Value : 0.9795
Prevalence : 0.4580
Detection Rate : 0.4468
Detection Prevalence : 0.4548
Balanced Accuracy : 0.9804

'Positive' Class : 0
```

Se puede observar que el nuevo modelo ha obtenido una sensibilidad del 97.55%, una especificidad del 98.52% y una precisión general de 98.08%. Estos valores son muy cercanos a la perfección, tal y como lo eran los obtenidos con el bosque original.

Para poder determinar cuál de los dos modelos es realmente mejor, se creará un **data frame** que permita comparar las métricas de ambos modelos.

Model	Accuracy	Sensitivity	Specificity
n = 50	0.9796	0.9746725	0.9837638
n = 100	0.9808	0.9755459	0.9852399

Table 10: Comparación del rendimiento de los modelos Random Forest con diferente número de árboles.

Por lo tanto, gracias a la comparación, se observa que no existen diferencias muy significativas entre los dos modelos. De hecho, las diferencias en sensibilidad, especificidad y precisión se han visto alteradas en un -0.2678571%, un 0% y un -0.1222494%, respectivamente.

4 Conclusión y discusión

En este análisis se evaluaron distintos modelos de clasificación, considerando métricas clave como precisión, sensibilidad y especificidad para determinar cuál ofrece el mejor desempeño en términos generales. Cada modelo tiene características particulares que explican su rendimiento y sus limitaciones. Para cada modelo, se ha escogido aquel que, debido a su configuración, parámetros y rendimiento, ha resultado más adecuado para nuestros datos.

Method	Model	Sensitivity	Specificity	Accuracy
k-NN	11 vecinos	0.5956	0.7343	0.6708
Naive Bayes	Sin Laplace	0.6725	0.6908	0.6824
ANN	1 capa	0.8689	0.9261	0.9377
SVM	Radial	0.9118	0.9542	0.9348
C5.0	Con boosting	0.9755	0.9845	0.9804
Random Forest	n = 50	0.9782	0.9845	0.9816

Table 11: Comparación de los modelos de clasificación según sus métricas de rendimiento.

5 Comparación de modelos de clasificación

El modelo de *k-Nearest Neighbors* (k-NN) alcanzó una precisión del 68.24%, una sensibilidad del 61.39% y una especificidad del 74.02%. Este algoritmo es uno de los más simples, basado únicamente en la idea de clasificar una instancia según la mayoría de las clases de sus vecinos más cercanos. Aunque es intuitivo y fácil de implementar, su desempeño es altamente dependiente de la elección de "k" y de la métrica de distancia utilizada. En este caso, "k=11" parece no haber sido suficiente para capturar correctamente las relaciones en los datos, lo que explica su baja sensibilidad y precisión general. Además, k-NN es especialmente útil en datos con relaciones lineales, por lo que limita su capacidad de predicción.

El modelo de *Naive Bayes* mostró el rendimiento más bajo, con una precisión del 64.32%, una sensibilidad del 53.01% y una especificidad del 73.87%. Este algoritmo asume que las variables predictoras son independientes entre sí, algo que rara vez ocurre en conjuntos de datos reales como el nuestro. Esta simplicidad lo hace rápido y eficiente, pero también limita su capacidad para capturar relaciones complejas. Además, se empleó el suavizado de Laplace, pero no resultó significativo en el aumento de la precisión.

Las *Redes Neuronales Artificiales* (ANN) son modelos inspirados en la estructura del cerebro humano, y están especialmente diseñados para capturar relaciones complejas entre las variables. En nuestro caso, con una sola capa logramos una precisión del 93.77%, una sensibilidad del 86.89% y una especificidad del 92.61%. Su buen desempeño puede atribuirse a su capacidad para adaptarse a patrones complejos en los datos. Sin embargo, el modelo mostró una sensibilidad más baja en comparación con *Random Forest* y *C5.0*, lo que indica una mayor tasa de falsos negativos. Esto puede deberse a una deficiencia en la capacidad de generalización de nuestra arquitectura, pero también refleja la necesidad de ajustes más precisos en los hiperparámetros para alcanzar su máximo potencial. A pesar de ello, sigue siendo una opción bastante competitiva.

El modelo de *Máquinas de Soporte Vectorial* (SVM) con kernel radial obtuvo una precisión del 85.72%, una sensibilidad del 83.75% y una especificidad del 87.38%. Este algoritmo es particularmente útil para problemas con fronteras de decisión no lineales, ya que utiliza un kernel para proyectar los datos en un espacio de mayor dimensión donde las clases sean más separables. Concretamente, el kernel radial causa una curvatura en la división que permite capturar patrones más complejos. Aun así, su desempeño no fue tan destacado en comparación con *Random Forest* o *C5.0*, posiblemente debido a la naturaleza del conjunto de datos, que podría no ajustarse perfectamente al enfoque de separación por márgenes máximos que emplea.

el SVM. Como medida de mejora, podrían probarse diferentes tipos de kernel o parámetros de regularización.

El modelo de **Árboles de decisión** *C5.0* con *boosting* mostró un desempeño comparable al de *Random Forest*, con una precisión del 98.04%, una sensibilidad del 97.55% y una especificidad del 98.45%. Debemos recordar que *C5.0* es una evolución de los árboles de decisión tradicionales, diseñada para manejar datos categóricos y numéricos de forma eficiente, lo cual resulta apropiado para nuestros datos. El uso del *boosting* ha mejorado el rendimiento al entrenar múltiples árboles secuenciales, logrando un modelo final más preciso y efectivo.

Finalmente, el modelo **Random Forest** se destacó como el más robusto, logrando una precisión del 98.32%, una sensibilidad del 97.55% y una especificidad del 98.97%. Este algoritmo combina múltiples árboles de decisión, obteniendo un modelo más estable y menos propenso al sobreajuste. El hecho de combinar votaciones de los árboles garantiza un desempeño consistente, sobre todo en datos con estructuras complejas. En especial, su alta especificidad, sensibilidad y precisión lo convierten en una herramienta ideal para minimizar los errores de clasificación. Aun así, debemos tener en cuenta que los *Random Forest* tienen un coste computacional elevado, lo que podría requerir mayores recursos en ciertas aplicaciones.

En conclusión, los modelos **Random Forest** y *C5.0* con *boosting* han resultado ser las opciones más recomendables debido a su excelente precisión, sensibilidad y especificidad, lo que los hace ideales para tareas críticas. Las *ANN* (una capa) representan una alternativa competitiva, aunque con menor sensibilidad, y los modelos de *SVM*, *k-NN* y *Naive Bayes* son menos adecuados para este problema debido a sus limitaciones inherentes y a su bajo desempeño relativo.

References

Mushroom. UCI Machine Learning Repository, 1981. DOI: <https://doi.org/10.24432/C5959T>.